

✓ Código Utilizado para Geração do JSON de Saíde e Log de Verossimilhança

```
import json
import math
import statistics
from collections import Counter, defaultdict
from dataclasses import dataclass
from datetime import datetime
from typing import Any, Dict, List, Optional, Tuple

def load_json(path: str) -> Dict[str, Any]:
    with open(path, 'r', encoding='utf-8') as f:
        return json.load(f)

# Esse código é o complemento natural do anterior: enquanto load_json lê um JSON,
# essa função salva dados em formato JSON em um arquivo.
def save_json(path: str, data: Dict[str, Any]) -> None:
    with open(path, 'w', encoding='utf-8') as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def to_float(value: Any) -> Optional[float]:
    # value: Any: aceita qualquer tipo de valor.
    # -> Optional[float]: pode retornar um float ou None (caso não consiga converter).
    if value is None or value == "":
        return None
    if isinstance(value, (int, float)): # Se já for número (int ou float), apenas converte para float (padroniza o tipo).
        return float(value)
    if isinstance(value, str): # Se for string, entra no processamento mais interessante
        clean = value.strip().replace("R$", "").replace(".", "").replace(",", ".")
        try:
            return float(clean)
        except ValueError:
            return None
    return None

def to_bool(value: Any) -> Optional[bool]:
    if value is None or value == "":
        return None
    if isinstance(value, bool): # Se já for booleano (True ou False), retorna direto.
        return value
    # Converte números: 0 -> False, qualquer outro número -> True
    if isinstance(value, (int, float)):
        return bool(value)
    if isinstance(value, str): # Se for string: remove espaços (strip) e converte para minúsculas (lower)
        text = value.strip().lower()
        if text in {"true", "1", "sim", "s", "yes", "y"}:
            return True
        if text in {"false", "0", "nao", "não", "n", "no"}:
            return False
        # Isso facilita comparar valores como "Sim", "SIM" etc.
    return None

# Essa função tenta interpretar uma data em diferentes formatos e padronizá-la para o formato ISO "YYYY-MM-DD".
# Se não conseguir converter, ela devolve o valor original (no caso de string) ou None.
def parse_date(value: Any) -> Optional[str]:
    # Recebe qualquer tipo (Any).
    # Retorna uma str (data formatada) ou None.
    if value is None or value == "":
        return None
    if isinstance(value, str): # Só tenta converter se o valor for uma string.
        for fmt in ("%Y-%m-%d", "%d/%m/%Y", "%Y/%m/%d", "%d-%m-%Y"):
            try:
                return datetime.strptime(value, fmt).strftime("%Y-%m-%d")
            except ValueError:
                continue
    return value
    return None

# Essa função é uma forma segura e concisa de calcular a média de uma lista de números, evitando erro quando a lista está vazia
def safe_mean(values: List[float]) -> Optional[float]:
    # Recebe uma lista de números (List[float]).
    # Retorna um float (a média) ou None se não houver dados.
    return statistics.mean(values) if values else None

def safe_median(values: List[float]) -> Optional[float]:
    return statistics.median(values) if values else None
```

```

def safe_mode(values: List[float]) -> Optional[float]:
    if not values: # Se a lista estiver vazia, retorna None.
        return None
    counts = Counter(values) # Usa Counter (da biblioteca collections) para contar quantas vezes cada valor aparece
    top_count = max(counts.values()) # Encontra a maior frequência (quantas vezes o valor mais comum aparece).
    # Cria uma lista com todos os valores que têm essa frequência máxima.
    # Isso resolve o problema de múltiplas modas.
    top_values = [k for k, v in counts.items() if v == top_count]
    return top_values[0] if top_values else None # se der empate, retorna apenas o primeiro valor da lista

def safe_variance(values: List[float]) -> Optional[float]:
    return statistics.variance(values) if len(values) > 1 else 0.0 if values else None

# idem anterior, porem calcula o esvio padrão
def safe_std(values: List[float]) -> Optional[float]:
    return statistics.stdev(values) if len(values) > 1 else 0.0 if values else None

# retorna mínimo
def safe_min(values: List[float]) -> Optional[float]:
    return min(values) if values else None

# retorna máximo
def safe_max(values: List[float]) -> Optional[float]:
    return max(values) if values else None

# retorna a amplitude
def amplitude(values: List[float]) -> Optional[float]:
    return (max(values) - min(values)) if values else None

def percentile(values: List[float], p: float) -> Optional[float]:
    if not values:
        return None
    ordered = sorted(values)
    if len(ordered) == 1:
        return ordered[0]
    idx = (len(ordered) - 1) * p
    lo = math.floor(idx)
    hi = math.ceil(idx)
    if lo == hi:
        return ordered[int(idx)]
    fraction = idx - lo
    return ordered[lo] + (ordered[hi] - ordered[lo]) * fraction

# Essa função calcula os quartis de um conjunto de dados usando a função percentile que você já viu.
def quartiles(values: List[float]) -> Dict[str, Optional[float]]:
    # Define uma função que calcula os quartis de uma lista de números
    # Retorna um dicionário com Q1, mediana (Q2) e Q3
    return {
        "q1": percentile(values, 0.25),
        # Primeiro quartil (25%)
        # 25% dos dados estão abaixo desse valor
        "q2_mediana": percentile(values, 0.50),
        # Segundo quartil (50%) = mediana
        # Divide os dados ao meio
        "q3": percentile(values, 0.75),
        # Terceiro quartil (75%)
        # 75% dos dados estão abaixo desse valor
    }

# Essa função calcula os decis, que são uma generalização dos quartis, dividindo os dados em 10 partes iguais.
def deciles(values: List[float]) -> Dict[str, Optional[float]]:
    # Define uma função que calcula os decis de uma lista de números
    # Retorna um dicionário com d1 até d9
    return {f"d{i}": percentile(values, i / 10) for i in range(1, 10)}

def percentiles_selected(values: List[float]) -> Dict[str, Optional[float]]:
    # Função que calcula alguns percentis específicos de uma lista de números
    # Retorna um dicionário com percentis escolhidos (10%, 25%, 50%, 75%, 90%)
    return {
        "p10": percentile(values, 0.10),
        # 10º percentil → 10% dos dados estão abaixo desse valor
        "p25": percentile(values, 0.25),
        # 25º percentil → também chamado de Q1 (primeiro quartil)
        "p50": percentile(values, 0.50),
        # 50º percentil → mediana
        "p75": percentile(values, 0.75),
        # 75º percentil → também chamado de Q3 (terceiro quartil)
        "p90": percentile(values, 0.90),
        # 90º percentil → valores altos da distribuição (topo dos dados)
    }

# Essa função cria uma tabela de frequência, ou seja: ela conta quantas vezes cada

```

```

# valor aparece e também calcula o percentual de ocorrência.
def frequency_table(values: List[Any], top_n: int = 10) -> List[Dict[str, Any]]:
    counts = Counter(v for v in values if v is not None and v != "")
    # Conta quantas vezes cada valor aparece na lista
    # Ignora None e strings vazias ""
    total = sum(counts.values())
    # Soma total de ocorrências (quantos itens válidos existem no total)
    table = []
    # Lista que vai armazenar o resultado final (a tabela)
    for item, count in counts.most_common(top_n):
        # Pega os top_n valores mais frequentes
        # Ex: ("banana", 5), ("maçã", 3)
        table.append({
            "valor": item,
            # O valor original (ex: "banana")
            "frequencia": count,
            # Quantas vezes ele aparece
            "percentual": round((count / total) * 100, 2) if total else 0.0,
            # Calcula o percentual que esse valor representa do total
            # Ex: 5 de 20 -> 25%
        })
    return table

# Essa função monta uma tabela de contingência, também chamada de tabela cruzada (cross-tab).
# Ela serve para contar relações entre duas variáveis categóricas.
def contingency_table(rows: List[Any], cols: List[Any], top_n: int = 10) -> Dict[str, Dict[str, int]]:
    result: Dict[str, Dict[str, int]] = defaultdict(lambda: defaultdict(int))
    # Cria um dicionário alinhado automaticamente inicializado com 0
    # Ex: result["A"]["X"] = 0 inicialmente
    for r, c in zip(rows, cols):
        # Percorre as duas listas ao mesmo tempo (par a par)
        if r is None or c is None or r == "" or c == "":
            # Ignora valores vazios ou inválidos
            continue
        result[str(r)][str(c)] += 1 # Conta quantas vezes o par (r, c) aparece
    # Limita para não explodir o JSON
    limited = {}
    # Novo dicionário para versão reduzida do resultado
    for i, (rk, rv) in enumerate(result.items()):
        # Percorre cada linha da tabela
        if i >= top_n:
            # Limita número de linhas
            break
        limited[rk] = dict(list(rv.items())[:top_n])
    # Converte a linha em dict normal e limita colunas também
    return limited

# Essa função calcula a correlação de Pearson, que mede o quanto duas variáveis
# estão relacionadas linearmente (se uma sobe, a outra sobe ou desce junto).
def pearson_correlation(x: List[float], y: List[float]) -> Optional[float]:
    # Função que calcula a correlação de Pearson entre duas listas
    if len(x) != len(y) or len(x) < 2:
        # As listas precisam ter o mesmo tamanho e pelo menos 2 elementos
        return None
    mx = safe_mean(x) # Média de x
    my = safe_mean(y) # Média de y
    sx = safe_std(x) # Desvio padrão de x
    sy = safe_std(y) # Desvio padrão de y
    if mx is None or my is None or sx in (None, 0) or sy in (None, 0):
        # Se alguma estatística não puder ser calculada ou for 0 (sem variação), não dá para calcular correlação
        return None
    numerator = sum((a - mx) * (b - my) for a, b in zip(x, y))
    # Parte principal da fórmula:
    # mede como x e y variam juntos em relação às médias
    denominator = (len(x) - 1) * sx * sy
    # Normaliza o valor usando desvio padrão e tamanho da amostra
    if denominator == 0: # Evita divisão por zero
        return None
    return numerator / denominator # Resultado final da correlação de Pearson

# Essa função faz uma normalização min-max, que é uma técnica para colocar todos os valores de uma lista na escala de 0 a 1.
def min_max_normalize(values: List[float]) -> List[Optional[float]]:
    # Função que normaliza valores para o intervalo [0, 1]
    if not values:
        # Se a lista estiver vazia, não há o que normalizar
        return []
    lo, hi = min(values), max(values) # Encontra o menor (lo) e o maior (hi) valor da lista
    if hi == lo: # Se todos os valores são iguais, não há variação
        return [0.0 for _ in values] # Retorna lista de zeros (todos iguais após normalização)
    return [(v - lo) / (hi - lo) for v in values]
    # Aplica fórmula de normalização min-max:
    # transforma cada valor para escala entre 0 e 1

```

```

# Essa função faz padronização (z-score), que é outra forma de "normalizar" dados – diferente do min-max que você viu antes.
def z_score_standardize(values: List[float]) -> List[Optional[float]]:
# Função que padroniza os valores usando z-score
# Resultado indica quantos desvios padrão cada valor está da média
    if not values: # Se a lista estiver vazia, não há o que calcular
        return []
    mu = safe_mean(values) # Calcula a média ( $\mu$ )
    sigma = safe_std(values) # Calcula o desvio padrão ( $\sigma$ )
    if mu is None: # Se não foi possível calcular a média, retorna vazio
        return []
    if sigma in (None, 0): # Se não há variação (desvio padrão zero), todos os valores são iguais
        return [0.0 for _ in values] # Nesse caso, todos recebem 0
    return [(v - mu) / sigma for v in values]
# Fórmula do z-score:
# (valor - média) / desvio padrão

# Essa função detecta outliers (valores fora do padrão) usando o método do IQR
# (Interquartile Range / Amplitude Interquartilica), que é muito comum em estatística.
def iqr_outliers(values: List[float]) -> Dict[str, Any]:
# Função que detecta outliers usando IQR
# Retorna quartis, limites e os valores fora do padrão
    if len(values) < 4: # Com poucos dados, o cálculo de quartis não é confiável
        return {
            "q1": percentile(values, 0.25), # Primeiro quartil (25%)
            "q3": percentile(values, 0.75), # Terceiro quartil (75%)
            "iqr": None, # Não dá para calcular bem o IQR
            "limite_inferior": None, # Sem limite inferior
            "limite_superior": None, # Sem limite superior
            "indices_outliers": [], # Nenhum outlier identificado
            "valores_outliers": [], # Nenhum valor fora do padrão
        }
    q1 = percentile(values, 0.25) # Calcula o primeiro quartil
    q3 = percentile(values, 0.75) # Calcula o terceiro quartil
    assert q1 is not None and q3 is not None # Garante que os valores não são None (checagem de segurança)
    iqr = q3 - q1 # IQR = intervalo entre Q3 e Q1 (parte central dos dados)
    low = q1 - 1.5 * iqr # Limite inferior para detectar outliers
    high = q3 + 1.5 * iqr # Limite superior para detectar outliers
    indices = [i for i, v in enumerate(values) if v < low or v > high] # Encontra índices dos valores fora dos limites
    out_vals = [values[i] for i in indices] # Pega os valores reais dos outliers
    return {
        "q1": q1, # resumo estatístico
        "q3": q3,
        "iqr": iqr,
        "limite_inferior": low, # limites para considerar outliers
        "limite_superior": high,
        "indices_outliers": indices, # posições dos outliers na lista
        "valores_outliers": out_vals, # valores que são outliers
    }

# Essa função gera um relatório de valores nulos (ou vazios) em uma lista de registros
# (dicionários). Ela conta quantos campos estão "faltando" em cada chave.
def null_report(records: List[Dict[str, Any]]) -> Dict[str, int]:
    counts: Dict[str, int] = Counter() # counts deve se comportar como dicionário com chaves do tipo str e valores do tipo
    for rec in records: # Percorre cada registro (cada "linha" de dados)
        for k, v in rec.items(): # Percorre cada campo (chave e valor) dentro do registro
            # k recebe a chave (key)
            # v recebe o valor (value) associado a essa chave
            if v is None or v == "": # Verifica se o valor está vazio (None ou string vazia)
                counts[k] += 1 # Incrementa 1 na contagem daquela chave
    return dict(counts) # Converte o Counter para dict normal e retorna
# dict(counts) pega o objeto counts (que no seu caso é um Counter) e converte para um dicionário comum (dict).

def infer_field_type(values: List[Any]) -> str:
# Função que tenta descobrir o tipo de uma coluna de dados
    not_null = [v for v in values if v is not None and v != ""] # Remove valores vazios (None ou "")
    if not not_null: # Se não sobrou nenhum valor válido
        return "indefinido"
    # Conta quantos valores podem ser interpretados como números
    # (diretamente ou via conversão com to_float)
    numeric = sum(1 for v in not_null if isinstance(v, (int, float)) or to_float(v) is not None)
    if numeric == len(not_null): # Se TODOS os valores são numéricos
        unique = len(set(float(to_float(v)) for v in not_null if to_float(v) is not None)) # Conta quantos valores numéricos
        # Um set guarda apenas valores únicos (remove duplicatas)
        if unique <= 12: # Poucos valores distintos → provavelmente discreto
            return "quantitativa_discreta"
        return "quantitativa_continua" # Muitos valores distintos → contínuo (ex: salário, peso)
    unique = len(set(str(v) for v in not_null)) # Se não for totalmente numérico, trata como texto e conta categorias única
    if unique <= 12: # Poucas categorias → variável categórica simples
        return "qualitativa_nominal"
    return "qualitativa_ordinal_ou_textual" # Muitas categorias → texto livre ou variável ordenada complexa

```

```

# mesmo código escrito de outra forma
# not_null = []
# for v in values:
#     if v is not None and v != "":
#         not_null.append(v)

# Essa função serve para arredondar números com segurança, sem quebrar o código quando o valor não for numérico.
def round_or_none(value: Optional[float], ndigits: int = 4) -> Optional[float]:
    # Função que arredonda um número, se ele for válido
    # Caso contrário, retorna o próprio valor (geralmente None)
    return round(value, ndigits) if isinstance(value, (int, float)) else value
    # Se value for int ou float -> arredonda
    # Senão (ex: None, string) -> retorna como está

# Essa função aplica arredondamento em toda uma lista de valores, usando a função round_or_none que você viu antes.
def round_list(values: List[Optional[float]], ndigits: int = 4) -> List[Optional[float]]:
    # Função que arredonda todos os elementos de uma lista
    # Mantém None ou valores não numéricos intactos
    return [round_or_none(v, ndigits) for v in values]
    # Para cada valor v na lista:
    # -> aplica round_or_none
    # -> retorna nova lista com valores arredondados

# Essa função calcula a log-verossimilhança (log-likelihood) de um conjunto de dados assumindo que eles seguem
# uma distribuição normal (gaussiana). Isso é um conceito mais avançado de estatística/probabilidade.
def normal_loglik(values: List[float]) -> Optional[float]:
    # Calcula a log-verossimilhança assumindo distribuição normal
    if not values: # Se não há dados, não dá para calcular
        return None
    mu = safe_mean(values) # Média dos valores ( $\mu$ )
    sigma = safe_std(values) # Desvio padrão ( $\sigma$ )
    if mu is None or sigma in (None, 0): # Sem média ou sem variação -> não dá para modelar normal
        return None # Soma da log-verossimilhança de cada ponto
    return sum(-0.5 * math.log(2 * math.pi * sigma ** 2) - ((x - mu) ** 2) / (2 * sigma ** 2) for x in values)

# Essa função calcula a log-verossimilhança (log-likelihood) assumindo que os dados seguem uma distribuição exponencial. É u
# “probabilística” da função anterior, mas para outro tipo de fenômeno.
def exponential_loglik(values: List[float]) -> Optional[float]:
    # Calcula a log-verossimilhança assumindo distribuição exponencial
    if not values or any(v <= 0 for v in values): # Se não há dados ou existe valor <= 0, não faz sentido na distribuição ex
        return None
    mu = safe_mean(values) # Calcula a média dos valores
    if mu in (None, 0): # Média inválida ou zero impede cálculo do parâmetro
        return None #  $\lambda$  (lambda) é o parâmetro da distribuição exponencial
        #  $\lambda = 1 / \text{média}$ 
    lam = 1 / mu
    return sum(math.log(lam) - lam * x for x in values)
    # Soma da log-verossimilhança para cada valor:
    #  $\log(\lambda) - \lambda x$ 

# Essa função calcula a log-verossimilhança (log-likelihood) assumindo que os dados seguem uma distribuição
# uniforme contínua – ou seja, todos os valores dentro de um intervalo têm a mesma probabilidade.
def uniform_loglik(values: List[float]) -> Optional[float]:
    # Calcula a log-verossimilhança assumindo distribuição uniforme
    if not values: # Se não há dados, não dá para calcular
        return None
    lo = min(values) # Menor valor da amostra (limite inferior do intervalo)
    hi = max(values) # Maior valor da amostra (limite superior do intervalo)
    if hi == lo: # Se todos os valores são iguais, não existe intervalo
        return None
    return len(values) * math.log(1 / (hi - lo))
    # Fórmula da log-verossimilhança da uniforme:
    # cada ponto tem probabilidade  $1 / (hi - lo)$ 
    # então somamos log dessa probabilidade para todos os pontos

# Essa função calcula a log-verossimilhança (log-likelihood) assumindo que os dados seguem uma
# distribuição de Poisson, que é usada para modelar contagens de eventos.
def poisson_loglik(counts: List[int]) -> Optional[float]:
    # Calcula log-verossimilhança assumindo distribuição de Poisson
    if not counts or any(c < 0 for c in counts): # Poisson só aceita contagens (0, 1, 2, 3...)
        return None
    lam = safe_mean([float(c) for c in counts])
    #  $\lambda$  (lambda) é a média das contagens
    # representa a taxa média de eventos
    if lam is None or lam <= 0: #  $\lambda$  precisa ser positivo
        return None
    total = 0.0 # acumulador da log-verossimilhança

```

```

    for k in counts:
        # para cada contagem observada
        total += k * math.log(lam) - lam - math.lgamma(k + 1)
        # fórmula da log-verossimilhança da Poisson:
        # log P(k | λ) = k log(λ) - λ - log(k!)
        # math.lgamma(k+1) = log(k!)
        # (forma estável numericamente)
    return total # retorna soma total da log-verossimilhança

# Essa função calcula a log-verossimilhança (log-likelihood) assumindo que os dados seguem uma distribuição de Bernoulli,
# que é o modelo mais simples de probabilidade: 0 ou 1 (falha/sucesso, sim/não, etc.).
def bernoulli_loglik(values: List[int]) -> Optional[float]: # Calcula log-verossimilhança assumindo distribuição de Bernoulli
    if not values or any(v not in (0, 1) for v in values): # Bernoulli só aceita valores 0 ou 1
        return None
    # Estima p (probabilidade de sucesso)
    # p = média dos 1s
    p = safe_mean([float(v) for v in values])
    if p is None or p in (0, 1): # p não pode ser 0 ou 1 porque geraria log(0)
        return None
    return sum(v * math.log(p) + (1 - v) * math.log(1 - p) for v in values)
    # log-verossimilhança da Bernoulli:
    # log P(x) = x log(p) + (1-x) log(1-p)

# Essa função calcula a log-verossimilhança (log-likelihood) assumindo que os dados seguem uma distribuição binomial,
# que generaliza a Bernoulli para casos em que há várias tentativas por observação.
def binomial_loglik(values: List[int], n: Optional[int] = None) -> Optional[float]:
    # Calcula log-verossimilhança assumindo distribuição binomial
    if not values or any(v < 0 for v in values): # Não aceita lista vazia nem valores negativos (não faz sentido em contagem)
        return None
    if n is None:
        # Se n (número de tentativas) não for fornecido, estima como o maior valor observado
        n = max(values) if values else 0
    if n <= 0:
        # n precisa ser positivo
        return None
    mean_val = safe_mean([float(v) for v in values]) # Média dos sucessos observados
    if mean_val is None:
        return None
    p = mean_val / n # Estima a probabilidade de sucesso p = média / número de tentativas
    if p <= 0 or p >= 1: # p precisa estar entre 0 e 1 (exclusivo)
        return None
    total = 0.0 # acumulador da log-verossimilhança
    for k in values: # k = número de sucessos observados
        if k > n: # não pode ter mais sucessos que tentativas
            return None
        total += math.lgamma(n + 1) - math.lgamma(k + 1) - math.lgamma(n - k + 1)
        # log do coeficiente binomial C(n, k)
        # log(n!) - log(k!) - log((n-k)!)
        total += k * math.log(p) + (n - k) * math.log(1 - p)
        # parte probabilística da binomial:
        # p^k * (1-p)^(n-k) em log
    return total

@dataclass
class PreparedData:
    negocio: Dict[str, Any]
    transacoes: List[Dict[str, Any]]
    dias: List[Dict[str, Any]]
    recepcao: Dict[str, Any]

# Esse trecho define uma classe chamada InsightCalculadoEngine, que é o
# “motor” responsável por processar dados e gerar insights.
class InsightCalculadoEngine:
    def __init__(self, data: Dict[str, Any]):
        # self representa a própria instância do objeto que está sendo criada ou usada
        self.raw = data # guarda os dados originais
        self.prepared = self._prepare_data(data) # chama o método da própria classe e salva o resultado em prepared

    # -----
    # Preparação
    # -----
    # Esse método _prepare_data é o coração do processamento de dados da sua classe. Ele pega
    # dados brutos e transforma em um formato limpo, padronizado e pronto para análise.
    def _prepare_data(self, data: Dict[str, Any]) -> PreparedData:
        negocio = data.get("negocio", {})
        dados = data.get("dados", {})
        recepcao = data.get("recepcao", {})
        # dict.get(chave, valor_padrao):
        # retorna o valor da chave se ela existir
        # se não existir, retorna o valor_padrao (em vez de dar erro)
        # se não existir, retorna {} (dicionário vazio)

        transacoes = [] # criando lista vazia
        for item in dados.get("transacoes", []): # [] retorna uma lista vazia

```



```

record = dict(item) # cria um dicionário (dict) a partir de item.
record["data"] = parse_date(record.get("data")) # trata o formato de data
if "valor" in record:
    record["valor"] = to_float(record.get("valor")) # trata float
if "pago_no_prazo" in record:
    record["pago_no_prazo"] = to_bool(record.get("pago_no_prazo")) # trata boolean
if "desconto" in record:
    record["desconto"] = to_float(record.get("desconto")) # trata float
if "marketing" in record:
    record["marketing"] = to_float(record.get("marketing")) # trata float
transacoes.append(record) # adicionando na lista

dias = [] # criando lista vazia
for item in dados.get("dias", []):
    record = dict(item)
    record["data"] = parse_date(record.get("data"))
    for field in ["receita", "despesa", "vendas_qtd", "clientes", "marketing", "desconto_medio"]:
        if field in record: # verifica se a chave field existe dentro do dicionário record
            record[field] = to_float(record.get(field)) # trata float
    dias.append(record)

if not dias and transacoes:
    dias = self._derive_daily_records(transacoes)

return PreparedData(negocio=negocio, transacoes=transacoes, dias=dias, recepcao=recepcao)

#Esse método _derive_daily_records é uma agregação de transações por dia. Ele pega dados "linha a linha" (transações) e
#transforma em um resumo diário estruturado – tipo o que você veria em um dashboard.
def _derive_daily_records(self, transacoes: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    by_day: Dict[str, Dict[str, Any]] = defaultdict(lambda: {
        "receita": 0.0,
        "despesa": 0.0,
        "vendas_qtd": 0,
        "clientes": set(),
        "marketing": 0.0,
        "desconto_medio_soma": 0.0,
        "desconto_medio_qtd": 0,
        "transacoes_qtd": 0,
    })

    for t in transacoes:
        day = t.get("data") or "sem_data"
        info = by_day[day]
        info["transacoes_qtd"] += 1
        valor = t.get("valor") or 0.0
        tipo = str(t.get("tipo", "")).lower()
        if tipo == "receita":
            info["receita"] += valor
            info["vendas_qtd"] += 1
        elif tipo == "despesa":
            info["despesa"] += valor
        info["marketing"] += (t.get("marketing") or 0.0)
        desconto = t.get("desconto")
        if desconto is not None:
            info["desconto_medio_soma"] += desconto
            info["desconto_medio_qtd"] += 1
        cliente = t.get("cliente")
        if cliente:
            info["clientes"].add(str(cliente))

    daily_records = []
    for day, info in sorted(by_day.items(), key=lambda x: x[0]):
        qtd_desc = info["desconto_medio_qtd"]
        daily_records.append({
            "data": day,
            "receita": round(info["receita"], 2),
            "despesa": round(info["despesa"], 2),
            "vendas_qtd": int(info["vendas_qtd"]),
            "clientes": int(len(info["clientes"])),
            "marketing": round(info["marketing"], 2),
            "desconto_medio": round((info["desconto_medio_soma"] / qtd_desc), 4) if qtd_desc else 0.0,
            "transacoes_qtd": int(info["transacoes_qtd"]),
        })
    return daily_records

# -----
# Séries auxiliares
# -----

# Esse método _series_from_days transforma os dados diários em séries numéricas separadas, ou seja,
# ele reorganiza o dataset para ficar pronto para análises estatísticas e gráficos (ou machine learning).
def _series_from_days(self, days: List[str]) -> Dict[str, List[float]]:

```

```

def _series_from_days(self) -> Dict[str, List[float]]:
    dias = self.prepared.dias # Obtém a lista de dias já preparados (provavelmente um array de dicionários com métricas
    receita = [d.get("receita") for d in dias if d.get("receita") is not None]
    # Cria uma lista com os valores de receita de cada dia
    # Ignora dias onde "receita" é None
    despesa = [d.get("despesa") for d in dias if d.get("despesa") is not None]
    # Cria uma lista com os valores de despesa
    # Também ignora valores None
    lucro = [] # Inicializa a lista de lucro (será calculada manualmente)
    for d in dias: # Itera sobre cada dia
        if d.get("receita") is not None and d.get("despesa") is not None:
            lucro.append(float(d["receita"]) - float(d["despesa"])) # Só calcula lucro se ambos os valores existirem
    vendas_qtd = [int(d.get("vendas_qtd") or 0) for d in dias]
    # Cria lista com quantidade de vendas por dia
    # Se não existir valor, usa 0
    # Converte tudo para inteiro
    clientes = [float(d.get("clientes") or 0) for d in dias]
    # Lista com número de clientes por dia
    # Usa 0 como padrão e converte para float
    marketing = [float(d.get("marketing") or 0) for d in dias]
    # Lista com valores de investimento em marketing
    # Também garante float e valor padrão 0
    desconto_medio = [float(d.get("desconto_medio") or 0) for d in dias]
    # Lista com desconto médio aplicado por dia
    # Usa 0 se não houver valor
    transacoes_qtd = [int(d.get("transacoes_qtd") or d.get("vendas_qtd") or 0) for d in dias]
    # Lista com quantidade de transações
    # Prioridade:
    # 1. "transacoes_qtd"
    # 2. "vendas_qtd" (fallback)
    # 3. 0 (se nenhum existir)
    return {
        "receita": receita,
        "despesa": despesa,
        "lucro": lucro,
        "vendas_qtd": vendas_qtd,
        "clientes": clientes,
        "marketing": marketing,
        "desconto_medio": desconto_medio,
        "transacoes_qtd": transacoes_qtd,
    }

# Esse método _ticket_values extrai uma lista de valores de "tickets" (valores de vendas) a partir
# das transações – mais especificamente, ele pega somente as receitas.
def _ticket_values(self) -> List[float]:
    tickets = [] # Inicializa uma lista vazia que armazenará os valores dos tickets (receitas)
    for t in self.prepared.transacoes: # Percorre cada transação presente na estrutura preparada
        if str(t.get("tipo", "")).lower() == "receita" and t.get("valor") is not None:
            # Verifica duas condições:
            # 1. O tipo da transação é "receita" (case insensitive)
            # - usa .get("tipo", "") para evitar erro se a chave não existir
            # - converte para string e minúsculo para padronizar comparação
            # 2. O campo "valor" não é None (garante que existe um valor válido)
            tickets.append(float(t["valor"])) # Converte o valor da transação para float e adiciona à lista
    return tickets # Retorna a lista contendo todos os valores de tickets encontrados

# -----
# Aulas
# -----

# Esse método aula_1 é basicamente uma função de "análise exploratória inicial" (EDA) do seu sistema.
# Ele organiza informações básicas do dataset para responder: "o que eu tenho de dados aqui?"
def aula_1(self) -> Dict[str, Any]: # Define o método "aula_1", que retorna um dicionário com análises iniciais dos da
    transacoes = self.prepared.transacoes # Extrai a lista de transações já preparadas
    dias = self.prepared.dias # Extrai a lista de registros diários já preparados
    if transacoes: # Verifica se existem transações
        keys = sorted({k for rec in transacoes for k in rec.keys()}) # sorted transforma o set em uma lista ordenad
        # Cria um conjunto com todas as chaves existentes nas transações
        # Depois converte em lista ordenada
        # Ex: ["valor", "data", "tipo", "cliente"]
        classificacao = {} # Inicializa dicionário que armazenará o tipo de cada campo
        for key in keys: # Itera sobre cada campo existente nas transações
            valores = [rec.get(key) for rec in transacoes] # Coleta todos os valores daquele campo em todas as transac
            classificacao[key] = infer_field_type(valores)
            # Usa uma função que infere o tipo do campo
            # Ex: quantitativa, qualitativa, etc.
        else: # Caso não existam transações
            classificacao = {} # Define classificação vazia

    sample_size = min(5, len(transacoes)) # Define tamanho da amostra (no máximo 5 registros)
    amostra = transacoes[:sample_size] # Pega os primeiros registros como amostra

    return { # Retorna um relatório estruturado com análises da "aula 1"

```



```

"tema": "Entender os dados do negócio", # Define o objetivo da análise
# Descreve o problema de negócio em linguagem simples
"problema_financeiro": "O empreendedor possui dados, mas não sabe o que está registrando nem como organizar isso"
"calculos": { # Seção com métricas e cálculos
    "populacao_transacoes": len(transacoes), # Quantidade total de transações
    "populacao_registros_diarios": len(dias), # Quantidade total de registros diários
    "amostra_exibida": sample_size, # Quantidade de registros mostrados na amostra
    "classificacao_campos_transacoes": classificacao, # Tipos de cada campo das transações
    "campos_faltantes_transacoes": null_report(transacoes), # Conta valores nulos/vazios nas transações
    "campos_faltantes_dias": null_report(dias), # Conta valores nulos/vazios nos dados diários
    "amostra_transacoes": amostra, # Exibe amostra dos dados
},
"insights": [ # Lista de interpretações automáticas
    # Explica o objetivo da análise de tipos
    "Nesta etapa o sistema identifica quais campos são qualitativos e quais são quantitativos.",
    # Explica que o sistema detecta problemas nos dados
    "Também aponta lacunas iniciais para preparar a análise exploratória das próximas aulas.",
],
]
}

def aula_2(self) -> Dict[str, Any]: # Define o método aula_2, que retorna um relatório estatístico do negócio

    # Converte os dados diários em séries numéricas (listas)
    # Ex: receita = [100, 200, 150, ...]

    series = self._series_from_days()
    tickets = self._ticket_values() # Extrai valores de vendas (receitas) para análise de ticket

    def resumo(values: List[float]) -> Dict[str, Optional[float]]: # Função interna que calcula estatísticas descritiva
        return {
            "media": round_or_none(safe_mean(values), 4), # Calcula a média dos valores e arredonda
            "mediana": round_or_none(safe_median(values), 4), # Calcula a mediana (valor central)
            "moda": round_or_none(safe_mode(values), 4), # Calcula a moda (valor mais frequente)
            "amplitude": round_or_none(amplitude(values), 4), # Diferença entre máximo e mínimo
            "variância": round_or_none(safe_variance(values), 4), # Mede dispersão dos dados
            "desvio_padrao": round_or_none(safe_std(values), 4), # Mede o quanto os valores variam em torno da média
            "minimo": round_or_none(safe_min(values), 4), # Menor valor da lista
            "maximo": round_or_none(safe_max(values), 4), # Maior valor da lista
        }

    return {# Retorna um relatório estruturado da aula 2
        "tema": "Descobrir o comportamento financeiro central do negócio", # Define o objetivo da análise
        # Explica o problema de negócio
        "problema_financeiro": "O empreendedor não sabe qual é o comportamento financeiro típico do negócio.",
        "calculos": { # Bloco com cálculos estatísticos principais
            "receita_diaria": resumo(series["receita"]), # Estatísticas da receita por dia
            "despesa_diaria": resumo(series["despesa"]), # Estatísticas da despesa por dia
            "lucro_diario": resumo(series["lucro"]), # Estatísticas do lucro por dia
            "ticket_receita": resumo(tickets), # Estatísticas dos valores de venda (ticket médio)
        },
        "insights": [ # Interpretações automáticas dos dados
            # Explica diferença entre medidas de tendência central
            "A média mostra o comportamento central, enquanto mediana e moda ajudam a validar se há distorções.",
            # Explica dispersão e estabilidade do negócio
            "A variância e o desvio padrão ajudam a medir estabilidade financeira.",
        ],
    ],
}

# Obtém percentis, quartis e decis a partir dos dados de entrada
def aula_3(self) -> Dict[str, Any]:
    transacoes = self.prepared.transacoes
    series = self._series_from_days()
    tickets = self._ticket_values()
    categorias = [t.get("categoria") for t in transacoes]
    formas_pagamento = [t.get("forma_pagamento") for t in transacoes]

    return {
        "tema": "Organizar os dados para entender distribuição e faixas de desempenho",
        "problema_financeiro": "O empreendedor vê números soltos, mas não enxerga faixas de valor, frequência ou concent
        "calculos": {
            "tabela_frequencia_categoria": frequency_table(categorias),
            "tabela_frequencia_forma_pagamento": frequency_table(formas_pagamento),
            "tabela_contingencia_categoria_pagamento": contingency_table(categorias, formas_pagamento),
            "quartis_receita_diaria": {k: round_or_none(v, 4) for k, v in quartiles(series["receita"]).items()},
            "quartis_lucro_diario": {k: round_or_none(v, 4) for k, v in quartiles(series["lucro"]).items()},
            "decis_ticket": {k: round_or_none(v, 4) for k, v in deciles(tickets).items()},
            "percentis_ticket": {k: round_or_none(v, 4) for k, v in percentiles_selected(tickets).items()},
        },
        "insights": [
            "As tabelas ajudam a descobrir onde há maior concentração de receitas, despesas e formas de pagamento.",
            "Quartis, decis e percentis permitem segmentar clientes, tickets e períodos do negócio.",
        ],
    },
}

```

```

def aula_4(self) -> Dict[str, Any]:
    series = self._series_from_days()
    corr_pairs = {
        "marketing_x_receita": pearson_correlation(series["marketing"], series["receita"]),
        "marketing_x_lucro": pearson_correlation(series["marketing"], series["lucro"]),
        "desconto_x_receita": pearson_correlation(series["desconto_medio"], series["receita"]),
        "clientes_x_receita": pearson_correlation(series["clientes"], series["receita"]),
        "vendas_qtd_x_receita": pearson_correlation([float(v) for v in series["vendas_qtd"]], series["receita"]),
        "despesa_x_lucro": pearson_correlation(series["despesa"], series["lucro"]),
    }
    rounded = {k: round_or_none(v, 4) for k, v in corr_pairs.items()}
    valid = {k: v for k, v in rounded.items() if v is not None}
    strongest = None
    if valid:
        strongest = max(valid.items(), key=lambda item: abs(item[1]))

    return {
        "tema": "Entender o que influencia o resultado financeiro",
        "problema_financeiro": "O empreendedor quer saber o que parece impactar faturamento, lucro ou estabilidade.",
        "calculos": {
            "correlacoes": rounded,
            "relacao_mais_forte": {
                "par": strongest[0],
                "correlacao": strongest[1],
            } if strongest else None,
            "nota_causalidade": "Correlação não prova causalidade; os resultados indicam associação estatística e devem",
        },
        "insights": [
            "Esta etapa ajuda a entender o que se move junto com receita ou lucro.",
            "Ela não substitui análise causal, mas orienta testes e decisões do projeto.",
        ],
    }

def aula_5(self) -> Dict[str, Any]:
    transacoes = self.prepared.transacoes
    series = self._series_from_days()
    dias = self.prepared.dias

    pagamentos = [t.get("pago_no_prazo") for t in transacoes if t.get("pago_no_prazo") is not None]
    atrasos = [not p for p in pagamentos]
    lucro_negativo = [1 if v < 0 else 0 for v in series["lucro"]]
    dias_com_venda = [1 if (d.get("vendas_qtd") or 0) > 0 else 0 for d in dias]

    prob_atraso = (sum(atrasos) / len(atrasos)) if atrasos else None
    prob_lucro_negativo = (sum(lucro_negativo) / len(lucro_negativo)) if lucro_negativo else None
    prob_dia_com_venda = (sum(dias_com_venda) / len(dias_com_venda)) if dias_com_venda else None

    return {
        "tema": "Trabalhar incerteza e risco financeiro",
        "problema_financeiro": "O empreendedor precisa decidir mesmo sem garantias, medindo chance e risco.",
        "calculos": {
            "espaco_amostral_dias": len(dias),
            "eventos_observados": {
                "dias_com_venda": sum(dias_com_venda),
                "dias_com_lucro_negativo": sum(lucro_negativo),
                "pagamentos_em_atraso": sum(atrasos) if atrasos else 0,
            },
            "probabilidade_empirica_atraso_pagamento": round_or_none(prob_atraso, 4),
            "probabilidade_empirica_lucro_negativo": round_or_none(prob_lucro_negativo, 4),
            "probabilidade_empirica_dia_com_venda": round_or_none(prob_dia_com_venda, 4),
            "explicacao": {
                "classica": "Depende de eventos equiprováveis previamente definidos.",
                "empirica": "Calculada a partir da frequência observada nos dados do negócio.",
                "subjativa": "Pode ser adicionada pelo especialista do negócio como expectativa gerencial.",
            },
        },
        "insights": [
            "Aqui o sistema deixa de ser apenas descritivo e começa a lidar com risco financeiro.",
            "As probabilidades empíricas ajudam na tomada de decisão sob incerteza.",
        ],
    }

def aula_6(self) -> Dict[str, Any]:
    series = self._series_from_days()
    tickets = self._ticket_values()
    vendas_qtd = series["vendas_qtd"]
    venda_ocorrencia = [1 if v > 0 else 0 for v in vendas_qtd]

    discrete_models = {
        "poisson": poisson_loglik(vendas_qtd),
        "binomial": binomial_loglik(vendas_qtd),
        "bernoulli": bernoulli_loglik(venda_ocorrencia)
    }

```

```

        bernoulli = bernoulli_loglik(venda_ocorrendia),
    }
    continuous_models = {
        "gaussiana": normal_loglik(tickets),
        "exponencial": exponential_loglik(tickets),
        "uniforme": uniform_loglik(tickets),
    }

    valid_discrete = {k: v for k, v in discrete_models.items() if v is not None}
    valid_continuous = {k: v for k, v in continuous_models.items() if v is not None}

    best_discrete = max(valid_discrete.items(), key=lambda item: item[1]) if valid_discrete else None
    best_continuous = max(valid_continuous.items(), key=lambda item: item[1]) if valid_continuous else None

    return {
        "tema": "Modelar cenários financeiros com distribuições probabilísticas",
        "problema_financeiro": "O empreendedor quer escolher o modelo probabilístico mais adequado ao comportamento do n",
        "calculos": {
            "comparacao_modelos_discretos": {k: round_or_none(v, 4) for k, v in discrete_models.items()},
            "modelo_discreto_mais_apropriado": {
                "nome": best_discrete[0],
                "score_loglik": round_or_none(best_discrete[1], 4),
            } if best_discrete else None,
            "comparacao_modelos_continuos": {k: round_or_none(v, 4) for k, v in continuous_models.items()},
            "modelo_continuo_mais_apropriado": {
                "nome": best_continuous[0],
                "score_loglik": round_or_none(best_continuous[1], 4),
            } if best_continuous else None,
            "criterio": "O modelo com maior log-verossimilhança é tratado como o mais aderente aos dados observados.",
        },
        "insights": [
            "Esta etapa compara modelos para escolher o mais coerente com contagens e valores do negócio.",
            "O resultado serve como base para simulações e projeções futuras.",
        ],
    }

}

if __name__ == "__main__": # se o arquivo estiver sendo executado diretamente

    engine = InsightCalculadoEngine({})
    dados_empresa = load_json("alt_exemplo_entrada_insight_calculado.json")
    engine = InsightCalculadoEngine(dados_empresa)
    resultado = engine.aula_6()
    print("\nTeste 6 - aula6")
    print(resultado)
    save_json("alt_teste_saida_aula6.json", resultado)

print("\nTeste: log-verossimilhança")

lista_normal = [
    10.5, 11.2, 9.8, 10.9, 11.5,
    10.1, 9.9, 10.7, 11.0, 10.3,
    10.8, 11.1
]

lista_exponential = [
    0.4, 1.2, 0.9, 2.1, 1.5,
    0.7, 3.0, 1.8, 0.6, 2.7,
    1.1, 0.3
]

lista_uniform = [
    2.5, 4.1, 6.7, 8.3, 1.9,
    5.5, 7.2, 3.8, 9.0, 0.6,
    4.9, 6.1
]

lista_poisson = [
    3, 5, 4, 6, 2,
    5, 7, 4, 3, 6,
    5, 4
]

lista_bernoulli = [
    1, 0, 1, 1, 0,
    1, 0, 1, 1, 0,
    1, 0
]

lista_binomial = [
    4, 5, 6, 3, 7,

```

```

5, 4, 6, 5, 7,
3, 5
]

# Dicionário com todas as listas
listas = {
    "Normal": lista_normal,
    "Exponencial": lista_exponencial,
    "Uniforme": lista_uniform,
    "Poisson": lista_poisson,
    "Bernoulli": lista_bernoulli,
    "Binomial": lista_binomial
}

# Dicionário com todas as funções
funcoes = {
    "normal": normal_loglik,
    "exponencial": exponencial_loglik,
    "uniforme": uniform_loglik,
    "poisson": poisson_loglik,
    "bernoulli": bernoulli_loglik,
    "binomial": binomial_loglik
}

# Testar cada lista em todas as distribuições
for nome_lista, dados in listas.items():

    print(f"\n===== Testando lista {nome_lista} =====")

    for nome_funcao, funcao in funcoes.items():

        try:
            log_ver = funcao(dados)
            print(f"{nome_funcao} = {log_ver}")

        except Exception as e:
            print(f"{nome_funcao} = ERRO -> {e}")

```

Teste 6 - aula6
{'tema': 'Modelar cenários financeiros com distribuições probabilísticas', 'problema_financeiro': 'O empreendedor quer escol

Teste: log-verossimilhança

```

===== Testando lista Normal =====
normal = -9.06228028900098
exponencial = -40.386718705865206
uniforme = -6.36753901274604
poisson = -25.457067687057076
bernoulli = None
binomial = -12.024467414757098

```

```

===== Testando lista Exponencial =====
normal = -15.056855924105971
exponencial = -15.675101496296596
uniforme = -11.919021276123402
poisson = -15.783793418453083
bernoulli = None
binomial = -14.72903706671911

```

```

===== Testando lista Uniforme =====
normal = -27.926051678914632
exponencial = -31.43265891944722
uniforme = -25.538780470191213
poisson = -28.326379321043433
bernoulli = None
binomial = -32.631620306801

```

```

===== Testando lista Poisson =====
normal = -20.952856057240744
exponencial = -30.04892876131529
uniforme = -19.313254949209202
poisson = -22.636787197572847
bernoulli = None
binomial = -20.900900782051473

```

```

===== Testando lista Bernoulli =====
normal = -8.56253923187433
exponencial = None
uniforme = 0.0
poisson = -10.77297550512881
bernoulli = -8.150319191898308
binomial = -8.150319191898308

```

```

===== Testando lista Binomial =====
normal = -20.114284402989796
exponencial = -31.313254949209206

```

```
uniforme = -16.635532333438686
poisson = -22.73214397038521
bernoulli = None
binomial = -19.963343146199637
```

✓ Representação do JSON de Entrada

```
# {
#   "negocio": {
#     "nome": "Mercado Central Prime",
#     "segmento": "Alimentação",
#     "cidade": "Campinas"
#   },
#   "recepcao": {
#     "origem": "landing_page",
#     "observacao": "Base simulada para análises financeiras e testes de indicadores"
#   },
#   "dados": {
#     "transacoes": [
#       {"id": "T001", "data": "2026-04-01", "tipo": "receita", "categoria": "Venda", "valor": 415.75, "cliente": "Cliente"},
#       {"id": "T002", "data": "2026-04-01", "tipo": "despesa", "categoria": "Estoque", "valor": 185.40, "forma_pagamento": "F"},
#       {"id": "T003", "data": "2026-04-02", "tipo": "receita", "categoria": "Venda", "valor": 367.90, "cliente": "Cliente"},
#       {"id": "T004", "data": "2026-04-02", "tipo": "despesa", "categoria": "Marketing", "valor": 95.00, "forma_pagamento": "F"},
#       {"id": "T005", "data": "2026-04-03", "tipo": "receita", "categoria": "Venda", "valor": 540.20, "cliente": "Cliente"},
#       {"id": "T006", "data": "2026-04-03", "tipo": "despesa", "categoria": "Frete", "valor": 72.30, "forma_pagamento": "F"},
#       {"id": "T007", "data": "2026-04-04", "tipo": "receita", "categoria": "Venda", "valor": 198.60, "cliente": "Cliente"},
#       {"id": "T008", "data": "2026-04-04", "tipo": "despesa", "categoria": "Estoque", "valor": 132.80, "forma_pagamento": "F"},
#       {"id": "T009", "data": "2026-04-05", "tipo": "receita", "categoria": "Venda", "valor": 615.00, "cliente": "Cliente"},
#       {"id": "T010", "data": "2026-04-05", "tipo": "despesa", "categoria": "Aluguel", "valor": 250.00, "forma_pagamento": "F"},
#       {"id": "T011", "data": "2026-04-06", "tipo": "receita", "categoria": "Venda", "valor": 720.45, "cliente": "Cliente"},
#       {"id": "T012", "data": "2026-04-06", "tipo": "despesa", "categoria": "Imposto", "valor": 155.90, "forma_pagamento": "F"},
#       {"id": "T013", "data": "2026-04-07", "tipo": "receita", "categoria": "Venda", "valor": 145.00, "cliente": "Cliente"},
#       {"id": "T014", "data": "2026-04-07", "tipo": "despesa", "categoria": "Frete", "valor": 120.00, "forma_pagamento": "F"},
#       {"id": "T015", "data": "2026-04-08", "tipo": "receita", "categoria": "Venda", "valor": 845.70, "cliente": "Cliente"},
#       {"id": "T016", "data": "2026-04-08", "tipo": "despesa", "categoria": "Estoque", "valor": 305.50, "forma_pagamento": "F"},
#       {"id": "T017", "data": "2026-04-09", "tipo": "receita", "categoria": "Venda", "valor": 495.30, "cliente": "Cliente"},
#       {"id": "T018", "data": "2026-04-09", "tipo": "despesa", "categoria": "Energia", "valor": 110.75, "forma_pagamento": "F"},
#       {"id": "T019", "data": "2026-04-10", "tipo": "receita", "categoria": "Venda", "valor": 910.00, "cliente": "Cliente"},
#       {"id": "T020", "data": "2026-04-10", "tipo": "despesa", "categoria": "Estoque", "valor": 355.20, "forma_pagamento": "F"},
#       {"id": "T021", "data": "2026-04-11", "tipo": "receita", "categoria": "Venda", "valor": 310.40, "cliente": "Cliente"},
#       {"id": "T022", "data": "2026-04-11", "tipo": "despesa", "categoria": "Comissao", "valor": 82.00, "forma_pagamento": "F"},
#       {"id": "T023", "data": "2026-04-12", "tipo": "receita", "categoria": "Venda", "valor": 1125.90, "cliente": "Cliente"},
#       {"id": "T024", "data": "2026-04-12", "tipo": "despesa", "categoria": "Estoque", "valor": 430.60, "forma_pagamento": "F"}
#     ]
#   }
# }
```

✓ Representação do JSON de Saída

```
# {
#   "tema": "Modelar cenários financeiros com distribuições probabilísticas",
#   "problema_financeiro": "O empreendedor quer escolher o modelo probabilístico mais adequado ao comportamento do negócio.",
#   "calculos": {
#     "comparacao_modelos_discretos": {
#       "poisson": -12.0,
#       "binomial": null,
#       "bernoulli": null
#     },
#     "modelo_discreto_mais_apropriado": {
#       "nome": "poisson",
#       "score_loglik": -12.0
#     },
#     "comparacao_modelos_continuos": {
#       "gaussiana": -84.9147,
#       "exponencial": -87.8819,
#       "uniforme": -82.6616
#     },
#     "modelo_continuo_mais_apropriado": {
#       "nome": "uniforme",
#       "score_loglik": -82.6616
#     }
#   },
#   "criterio": "O modelo com maior log-verossimilhança é tratado como o mais aderente aos dados observados.",
# },
#   "insights": [
#     "Esta etapa compara modelos para escolher o mais coerente com contagens e valores do negócio.",
#     "O resultado serve como base para simulações e projeções futuras."
#   ]
# }
```

```
# ]  
# }
```